

2D Streamfunction-Vorticity CFD Solver

Technical Documentation

1. Problem Statement

1.1 Physical Problem

This solver simulates incompressible, viscous fluid flow in a 2D rectangular channel domain. The physical configuration consists of:

- **Domain dimensions:** Length $L = 10$ m, Height $H = 1$ m
- **Flow characteristics:** Inlet velocity $U_0 = 1$ m/s, Reynolds number $Re = 1000$
- **Flow type:** Laminar channel flow with parabolic inlet profile

The problem represents a classic computational fluid dynamics test case used to validate numerical methods for incompressible Navier-Stokes equations.

1.2 Governing Equations

The incompressible Navier-Stokes equations in 2D are reformulated using the streamfunction-vorticity (ψ - ω) approach:

Vorticity Transport Equation:

$$\frac{\partial \omega}{\partial t} + u \cdot \frac{\partial \omega}{\partial x} + v \cdot \frac{\partial \omega}{\partial y} = (1/Re) (\partial^2 \omega / \partial x^2 + \partial^2 \omega / \partial y^2)$$

Streamfunction-Vorticity Relation:

$$\nabla^2 \psi = \omega$$

or equivalently:

$$\partial^2 \psi / \partial x^2 + \partial^2 \psi / \partial y^2 = \omega$$

Velocity-Streamfunction Relations:

$$u = \partial \psi / \partial y$$

$$v = -\partial \psi / \partial x$$

where: - ω = vorticity ($\partial v / \partial x - \partial u / \partial y$) - ψ = streamfunction - u, v = velocity components in x and y directions - Re = Reynolds number ($\rho UL / \mu$)

1.3 Advantages of Streamfunction-Vorticity Formulation

This formulation offers several benefits: - **Automatic continuity satisfaction:** The continuity equation ($\nabla \cdot \mathbf{u} = 0$) is automatically satisfied by the velocity-streamfunction relationship - **No pressure term:** Eliminates the pressure field from the momentum equations, reducing computational complexity - **Natural boundary conditions:** Wall conditions are naturally expressed in terms of streamfunction values

2. Numerical Method

2.1 Discretization Approach

The solver employs a **finite difference method** with: - **Spatial discretization:** Second-order central differences - **Temporal discretization:** Explicit forward Euler for vorticity transport - **Grid:** Uniform rectangular mesh with $n_x \times n_y$ points

2.2 Grid Generation

The computational domain is discretized uniformly: - x -direction: $n_x = 201$ points, spacing $dx = L / (n_x - 1)$ - y -direction: $n_y = 51$ points, spacing $dy = H / (n_y - 1)$

$$x \in [0, L], \text{ with } dx = 0.05 \text{ m}$$

$$y \in [0, H], \text{ with } dy = 0.02 \text{ m}$$

2.3 Solution Algorithm

The solver uses a **segregated approach** with operator splitting:

Time-stepping loop: 1. **Solve Poisson equation for ψ :** Given ω , solve $\nabla^2\psi = \omega$ iteratively
 2. **Compute velocities:** Calculate u and v from ψ using finite differences
 3. **Advance vorticity:** Update ω using the vorticity transport equation
 4. **Apply boundary conditions:** Update boundary values for both ψ and ω
 5. **Check convergence:** Monitor residuals and iterate

3. Solution Methods

3.1 Streamfunction Solver (Poisson Equation)

The Poisson equation $\nabla^2\psi = \omega$ is solved using **Successive Over-Relaxation (SOR)** with point-wise Gauss-Seidel iteration.

Discretized equation:

$$\psi_{i,j} = [dx^2 (\psi_{i,j+1} + \psi_{i,j-1}) + dy^2 (\psi_{i+1,j} + \psi_{i-1,j}) + dx^2 dy^2 \omega_{i,j}] /$$

Algorithm: - Iterative method with relaxation parameter $\omega_{\text{relax}} = 1.0$ (Gauss-Seidel) -
 Convergence tolerance: 10^{-8} - Maximum iterations: 3000 - Updates are performed in-place for faster convergence

Why this method? - Simple to implement and parallelize - Robust convergence for elliptic problems - Well-suited for rectangular domains - Low memory requirements

3.2 Vorticity Advection-Diffusion Solver

The vorticity transport equation is advanced using **explicit Euler time integration** with

central differences for spatial derivatives.

Discretization:

Advection term:

$$\begin{aligned} \text{ADV} &= u \cdot \partial \omega / \partial x + v \cdot \partial \omega / \partial y \\ &\approx u_{ij} (\omega_{i+1,j} - \omega_{i-1,j}) / (2dx) + v_{ij} (\omega_{i,j+1} - \omega_{i,j-1}) / (2dy) \end{aligned}$$

Diffusion term:

$$\begin{aligned} \text{DIFF} &= (1/\text{Re}) \nabla^2 \omega \\ &\approx (1/\text{Re}) [(\omega_{i+1,j} - 2\omega_{ij} + \omega_{i-1,j}) / dx^2 + (\omega_{i,j+1} - 2\omega_{ij} + \omega_{i,j-1}) / dy^2] \end{aligned}$$

Time advancement:

$$\omega^{n+1}_{ij} = \omega^n_{ij} + \Delta t (-\text{ADV} + \text{DIFF})$$

Why explicit method? - Straightforward implementation - No need for matrix solvers - Suitable for moderate CFL numbers - Time step automatically adjusted for stability

4. Stability Analysis

4.1 CFL Condition (Advection)

For the advection term, the **Courant-Friedrichs-Lewy (CFL)** condition must be satisfied:

$$\text{CFL} = |u| \Delta t / \Delta x + |v| \Delta t / \Delta y \leq C_{\text{max}}$$

where $C_{\text{max}} \approx 1$ for stability.

Implemented constraint:

$$\Delta t_{\text{conv}} = 0.5 \times \min(\Delta x, \Delta y) / \max(u_0, 10^{-8})$$

This ensures $\text{CFL} \approx 0.5$, providing a safety margin.

4.2 Diffusion Stability

For the diffusion term with explicit Euler, the **von Neumann stability** condition requires:

$$D_{\text{max}} = (\Delta t / \text{Re}) \times (1/\Delta x^2 + 1/\Delta y^2) \leq 1/2$$

Implemented constraint:

$$\Delta t_{\text{diff}} = 0.25 \times \min(\Delta x^2, \Delta y^2) \times \text{Re}$$

This ensures the diffusion number $D \leq 0.25$, providing stability with margin.

4.3 Final Time Step Selection

The code automatically selects:

$$\Delta t = \min(\Delta t_{\text{conv}}, \Delta t_{\text{diff}})$$

For default parameters: - $\Delta t_{\text{conv}} = 0.5 \times 0.02 / 1.0 = 0.01 \text{ s}$ - $\Delta t_{\text{diff}} = 0.25 \times 0.0004 \times 1000 = 0.1 \text{ s}$ - **Selected $\Delta t = 0.01 \text{ s}$** (limited by convection)

5. Initial Conditions

5.1 Streamfunction Initialization

The streamfunction is initialized with a **fully developed parabolic profile** to accelerate convergence:

Inlet velocity profile:

$$u_{\text{inlet}}(y) = 6U_0 (y/H) (1 - y/H)$$

This parabolic profile satisfies: - No-slip at walls: $u(0) = u(H) = 0$ - Mean velocity: $\int u \, dy / H = U_0$

Streamfunction from velocity:

$$\psi(y) = \int_0^y u_{\text{inlet}}(\eta) \, d\eta$$

Computed using trapezoidal integration:

$$\psi_j = \psi_{j-1} + 0.5 (u_j + u_{j-1}) (y_j - y_{j-1})$$

Applied everywhere: The inlet profile is extended throughout the domain as the initial guess, which represents a reasonable starting point.

5.2 Vorticity Initialization

Initial vorticity is set to **zero in the interior** and then boundary conditions are applied immediately before the first iteration.

6. Boundary Conditions**6.1 Streamfunction Boundary Conditions****Bottom and Top Walls ($y = 0, H$):**

$$\psi(x, 0) = 0$$

$$\psi(x, H) = \psi_{\text{inlet}}(H) = \text{constant}$$

These ensure no flow through walls ($u \cdot n = 0$).

Inlet ($x = 0$):

$$\psi(0, y) = \psi_{\text{inlet}}(y)$$

Prescribed parabolic profile.

Outlet ($x = L$): Streamfunction naturally advects out; typically:

$$\partial\psi/\partial x = 0 \quad (\text{Neumann condition})$$

6.2 Vorticity Boundary Conditions

Bottom and Top Walls ($y = 0, H$): Wall vorticity enforces no-slip condition using **Thom's formula**:

$$\begin{aligned}\omega(x, 0) &= -2[\psi(x, 1) - \psi(x, 0)] / dy^2 \\ \omega(x, H) &= -2[\psi(x, ny-2) - \psi(x, H)] / dy^2\end{aligned}$$

This ensures the velocity gradient at the wall matches the no-slip requirement.

Inlet ($x = 0$): Vorticity consistent with parabolic inlet profile:

$$\omega_{\text{inlet}}(y) = -du_{\text{inlet}}/dy$$

For the parabolic profile:

$$\omega_{\text{inlet}}(y) = -6U_0/H[1 - 2y/H]$$

Computed using `np.gradient()` for accuracy.

Outlet ($x = L$): Convective boundary condition to allow vortices to exit smoothly:

$$\partial\omega/\partial t + U_{\text{local}} \cdot \partial\omega/\partial x = 0$$

Discretized using upwind differences:

$$\omega(L, y)^{n+1} = \omega(L-dx, y)^n - (U_{\text{local}} \cdot \Delta t / \Delta x) [\omega(L-dx, y) - \omega(L-2dx, y)]$$

where $U_{\text{local}} = u(L-dx, y)$ is the local velocity.

Corner Points: Averaged from adjacent boundaries:

$$\omega(0, 0) = 0.5 [\omega(1, 0) + \omega(0, 1)]$$

and similarly for other corners.

7. Convergence Criteria

7.1 Inner Iteration (Poisson Solver)

The streamfunction solver iterates until:

$$\max |\psi^{n+1} - \psi^n| < \text{tol}_{\psi} = 10^{-8}$$

7.2 Outer Iteration (Time Stepping)

The simulation continues until vorticity field converges:

$$||\omega^{n+1} - \omega^n||_2 / (||\omega^n||_2 + \varepsilon) < \text{tol}_{\omega} = 10^{-6}$$

where $\varepsilon = 10^{-14}$ prevents division by zero.

This indicates the solution has reached steady-state.

8. Output and Post-Processing

8.1 Solution Files

Output is written in **Tecplot ASCII format** at specified intervals: - Files:

`solution_001.dat` , `solution_002.dat` , ... - Contains: x , y , ψ , ω , u , v at all grid points - Frequency: Every 10 iterations (default)

8.2 Residual Logging

Convergence history is logged to `omegarsd.dat` :

Iter	Time[s]	OmegaResidual
1	0.0100	1.234567e-02
2	0.0200	9.876543e-03
...		

8.3 Final Output

Upon convergence or reaching maximum iterations: - `solution_final.dat` : Final converged solution - Console output: Convergence message and timing statistics

9. Computational Performance

9.1 Complexity

- **Per iteration:** $O(n_x \times n_y \times n_{\text{inner}})$
- **Total:** $O(n_{\text{outer}} \times n_x \times n_y \times n_{\text{inner}})$

For default parameters: - Grid: $201 \times 51 = 10,251$ points - Typical iterations: ~100-500 outer, ~100-500 inner per outer - Runtime: Minutes on modern CPU

9.2 Optimization Opportunities

Current implementation priorities: 1. **Code clarity** over raw performance 2. **Stability** over speed 3. **Extensibility** for future enhancements

Potential improvements: - Vectorization using NumPy operations - Multigrid methods for Poisson solver - Implicit time stepping for larger time steps - Parallel processing (OpenMP/MPI)

10. Validation and Testing

10.1 Expected Results

For the channel flow configuration: - Parabolic velocity profile should develop and maintain - Vorticity concentrated near walls - Steady-state reached after transient development

10.2 Key Dimensionless Parameters

- **Reynolds number ($Re = 1000$)**: Laminar flow regime
- **Aspect ratio ($L/H = 10$)**: Long channel allows development
- **Grid resolution**: Adequate for $Re = 1000$

10.3 Physical Consistency Checks

- Mass conservation: $\int u \, dy = \text{constant}$
 - Maximum velocity: $u_{\text{max}} \approx 1.5 U_0$
 - Wall vorticity: Proportional to wall shear stress
-

11. Usage Example

```
# Run with default parameters
python solver.py
```

```
# Custom grid and Reynolds number
python solver.py --nx 301 --ny 101 --Re 500

# Finer time step
python solver.py --dt 0.005 --itmax 5000

# More frequent output
python solver.py --outfreq 5
```

12. References and Further Reading

- 1. **Streamfunction-Vorticity Methods:** Classical approach for 2D incompressible flow
- 2. **Finite Difference Methods for PDEs:** Theoretical foundation
- 3. **Computational Fluid Dynamics:** Numerical techniques and stability analysis
- 4. **Thom's Formula:** Boundary vorticity calculation for no-slip walls

Appendix: Key Parameters Summary

Parameter	Default	Description
nx	201	Grid points in x
ny	51	Grid points in y
L	10.0 m	Domain length
H	1.0 m	Domain height
Re	1000	Reynolds number

U_0	1.0 m/s	Inlet velocity scale
Δt	auto	Time step (stability-limited)
tol_ ψ	10^{-8}	Poisson solver tolerance
tol_ ω	10^{-6}	Convergence tolerance
itmax	2000	Maximum iterations

Document Version: 1.0

Date: October 2025

Software: Python 3.x with NumPy